

EC 504
Fall, 2025
HW 3 Software

Due Friday, Nov. 14, 2025, 11:59 pm

1. **Finding Shortest Paths** In this problem, you will implement five versions of shortest path algorithms: a simple version of Dijkstra's algorithm using a list instead of a priority queue to find the nearest vertex at each iteration, a version of Dijkstra's algorithm that uses a binary heap as a priority queue, a simple version of Bellman-Ford that loops over vertices as described in the book version, a version of Bellman-Ford using a work queue implemented with the C++ standard queue, and a version of Bellman-Ford implemented with your own work queue. Actually, three of these algorithms are already implemented in the provided software. You have to implement two of them: Dijkstra with a priority queue and a version of Bellman-Ford with your own work queue.

I've provided three files: `Shortest_paths.cpp` is the main program with all the timing routines included. It reads a directed graph from a file. It loads the information into an array of vertices called `Nodes`. Associated with each node is a linked list of outgoing edges (arcs). The `Node` structure and the `arc` structure are defined in an include file `shortPath.h`, along with the five shortest path algorithms, two of which you need to develop. The five algorithms are:

- **Dijkstra**: This algorithm is provided, as a simple implementation of Dijkstra's algorithm with no data structures for finding the smallest distance node. The code simply searches the list of nodes to identify the one with the smallest distance. This algorithm is implemented in full so you can see how to use the graph data structures.
- **DijkstraHeap**: You are to implement this algorithm, adding a priority queue (a min-heap) using the include `myHeap.h`. This priority queue is implemented storing pointers to structures, and has all the needed functions to upheap when distances are decreased, and to extract minimum.
- **BellmanFord**: This algorithm is provided, and is a simple implementation of Bellman Ford with no data structures. We simply iterate over every edge updating distances and predecessors, and do it for at most number-of-nodes iterations. There is an optimization included: if an iteration over all the edges does not reduce any distance, then the algorithm stops and exits with the optimal shortest paths. The algorithm often exits after 10 or so full edge iterations, not 30000. Hence, it is not the stupidest implementation that the book suggests; that would try my patience!
- **BellmanFord_stdQueue**: This algorithm is provided, and uses a work queue to keep track of which nodes to scan. This algorithm is implemented using the C++ standard queue library. The algorithm exits when the work queue is completed. Notice how `node.position` is used to indicate whether a node is already in the work queue.
- **BellmanFord_Queue**: You are to implement this algorithm, by replacing the standard queue in the previous algorithm with your own queue. I suggest some form of array for this. Try not to do a ton of data movement. Try to be faster than the standard library implementation.

A makefile is included. To compile your executable, simply type `make shortest`. Three different input graphs are provided: `Graph1.txt`, `Graph2.txt`, `Graph3.txt`. A fourth input graph, `smallgraph.txt`, is provided along with the output file `smallgraph1.txt_out` to help you with debugging.

Note that the vertices in the input files are numbered from 1 to Nm . The `Nodes[]` data structure hence ignores the `Nodes[0]` element, so that the index in the array equals to the node id. The element `Nodes[0]` has no information, and should not be used in any of the algorithms.

To run the program, simply type `./shortest`. This will generate the output `smallgraph.txt_out` which you can compare with the output file provided. Note that the input file is hard-coded in the main program. This is so that you can run your code in interactive debuggers (e.g. VS Code, Visual Studio, ...) without having to jump through hoops to allow console input! Hence, you will have to edit the main program and recompile to change input files.

You are to run the executable `shortest` against the three graph inputs `Graph1.txt`, `Graph2.txt` and `Graph3.txt`. To do this, you have to edit the main file `Shortest_paths.cpp` and change the name of the input file to the input file you want. You will turn in a copy of all your include files, and the three output files `Graph1.txt_out`, `Graph2.txt_out` and `Graph3.txt_out`.

You will also turn in a pdf file with a discussion of your results, comparing the impact of using different data structures and algorithms on the runtimes for the three cases.

Note that you can compile and run the codes as they are and get the correct output for the algorithm that are already coded. You can use this to verify that your other algorithms are also getting correct outputs, as all five algorithms should find the same shortest paths.

If you program your algorithms well, your BellmanFord and DijkstraHeap algorithms should run much faster than the simple Dijkstra algorithm I've provided, for `Graph3.txt` that has 100,000 vertices. This is due to the $O(|V|^2)$ complexity versus the $O(|E| \log(|V|))$ complexity of Dijkstra's algorithm with a binary heap. The surprise is the Bellman Ford algorithms' speed, which shows that an average case for this algorithm can be very fast when you avoid unneeded overhead.